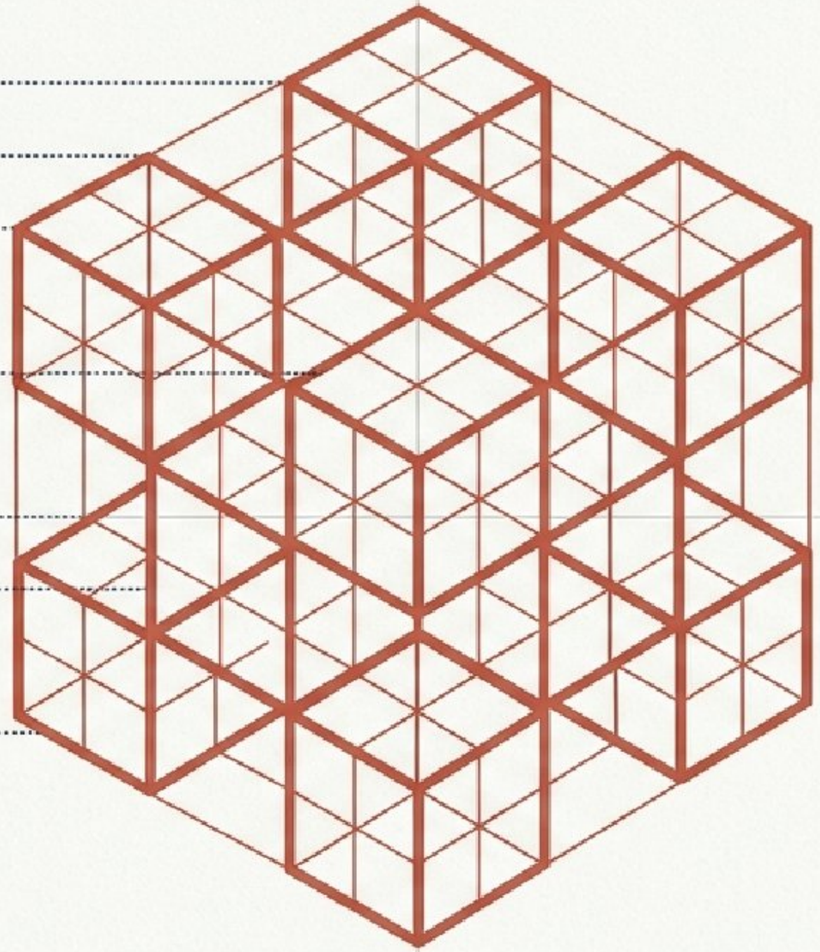


HANDWRITTEN DIGIT RECOGNITION USING CNN (Convolutional Neural Network)

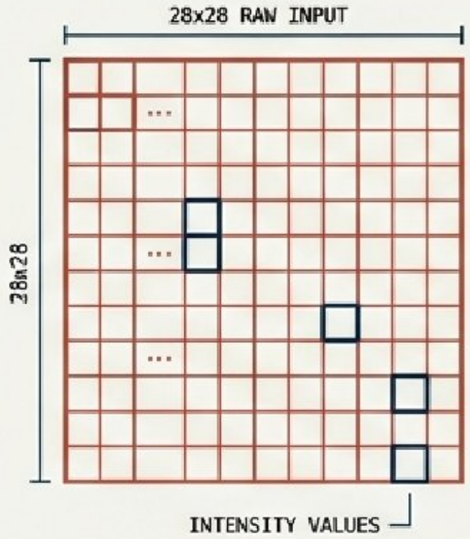
Implementation Guide: The Journey of a Digit

The Manufacturing Pipeline

Phase A: The Fuel

Step 1: Load & Preprocess

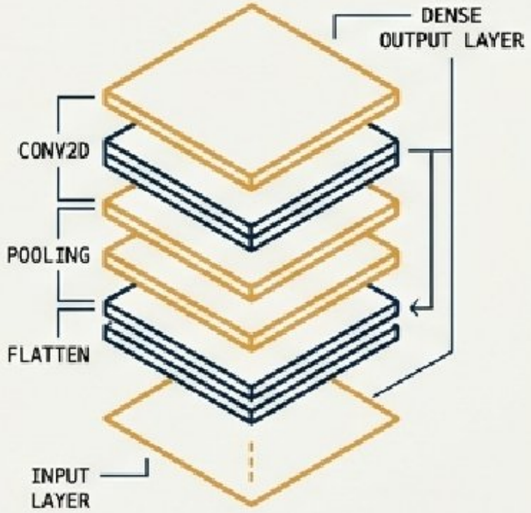
Preparing raw pixels for the engine.



Phase B: The Engine

Step 2: Build Architecture

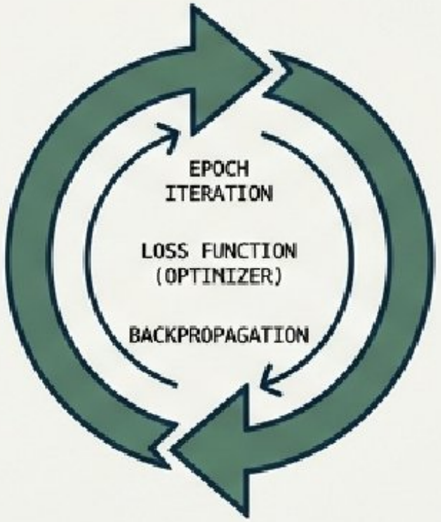
Constructing the feature extractors.



Phase C: The Loop

Steps 3 & 4: Compile & Train

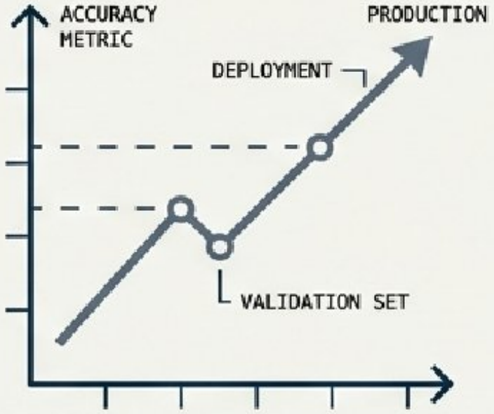
Defining the rules and iterating over data.



Phase D: The Dashboard

Steps 5, 6 & 7: Evaluate & Predict

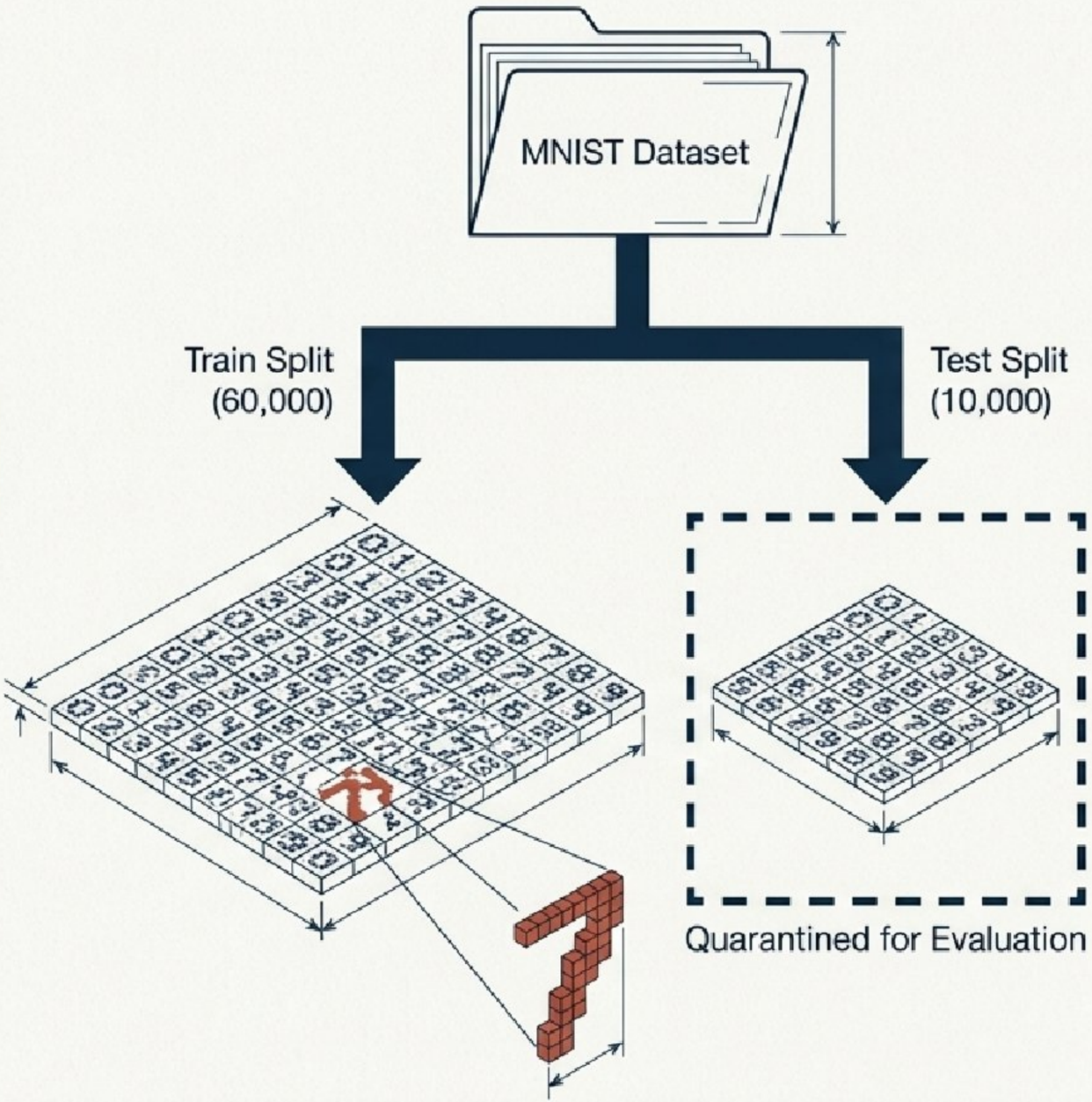
Measuring accuracy and deploying into the wild.



Ingesting the Raw Material

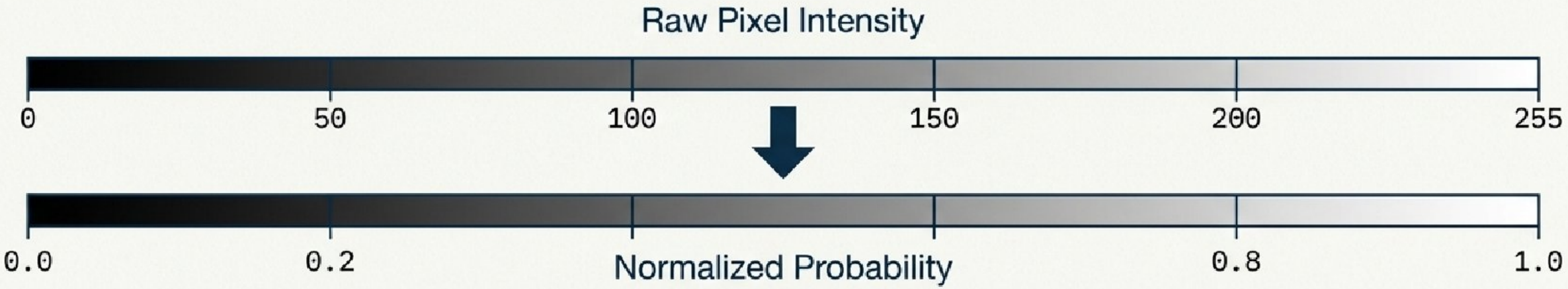
The model requires both raw material (images) and an answer key (labels). The dataset is split immediately to ensure the AI is eventually tested on images it has never seen before.

```
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```



Math to Mechanics: Normalization

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```



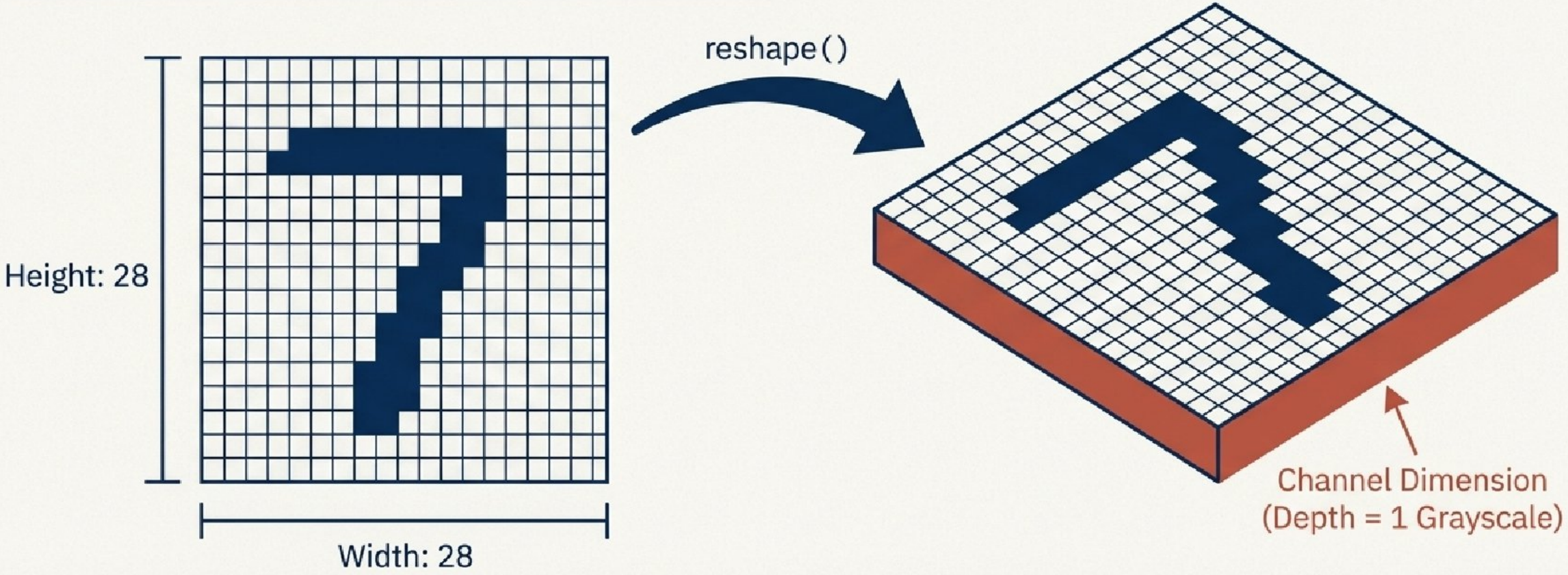
The Data Transformation Matrix

Dimension	Raw State	Processed State
Value Range	Integers: `0 to 255`	Floats: `0.0 to 1.0`
Meaning	Absolute screen brightness	Relative mathematical weight
Purpose	Human screen rendering	Stable gradient descent for the AI

The Tensor Explainer: Adding Volume

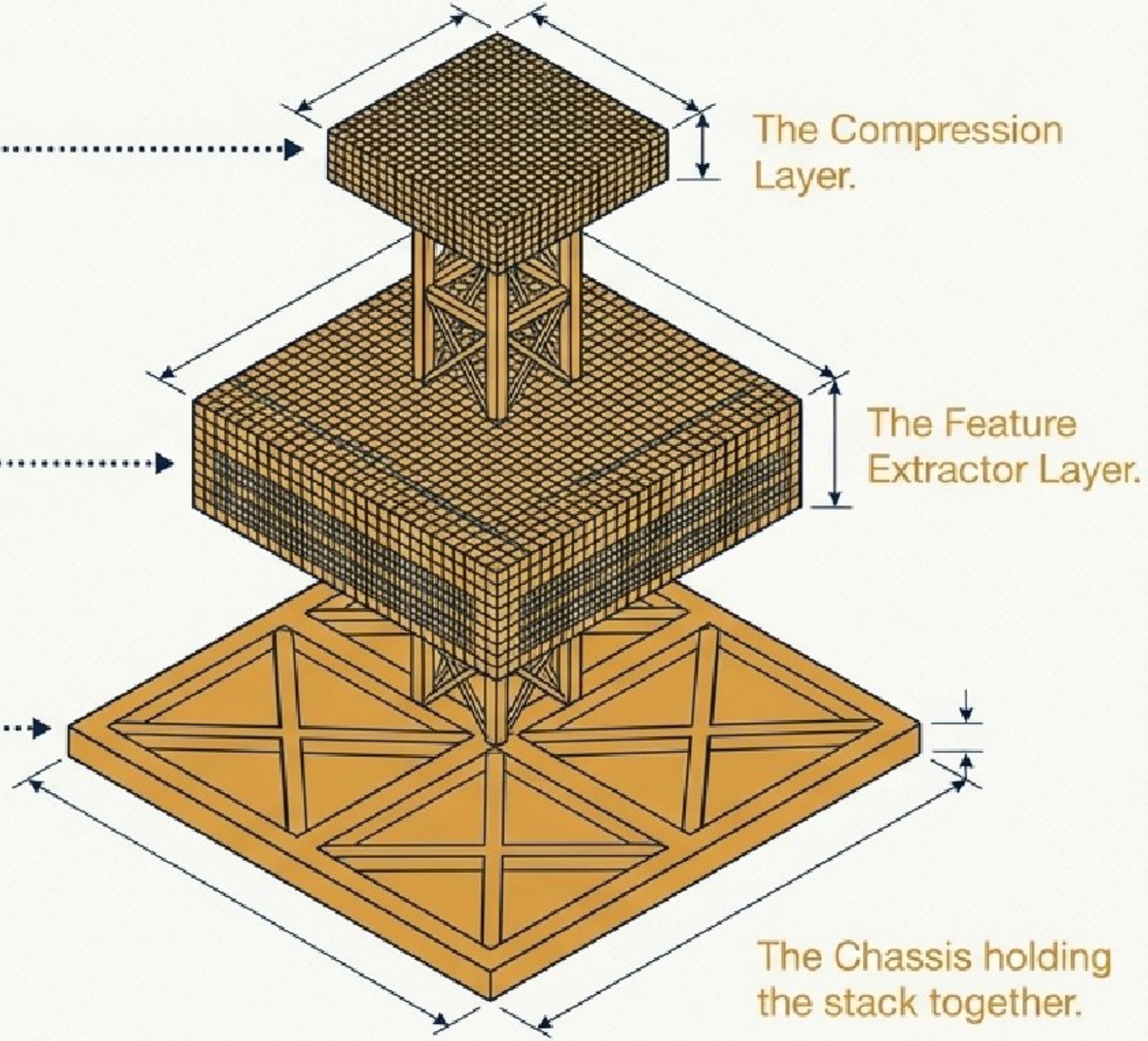
Convolutional networks expect images to have physical volume (Height × Width × Color Channels). Because MNIST is grayscale, we explicitly define the depth as 1. If it were an RGB photo, the depth would be 3.

```
x_train = x_train.reshape(-1, 28, 28, 1)
```



Constructing the Architecture

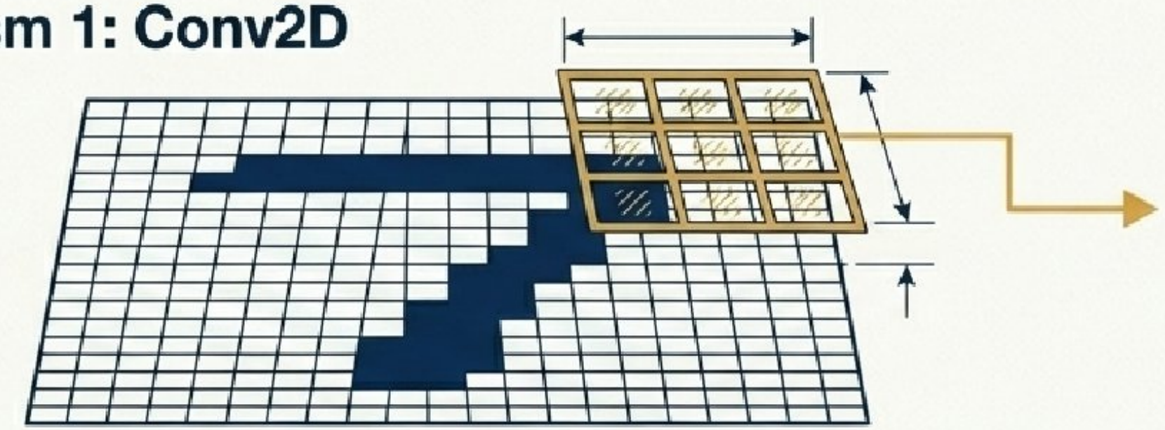
```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
])
```



Micro-Mechanics: Conv & Pool

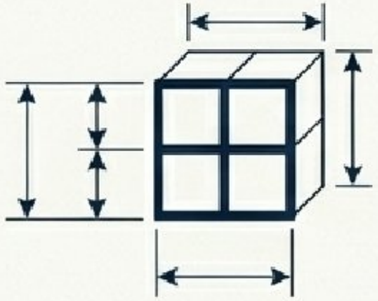
The network isolates geometric features in patches before condensing the map.

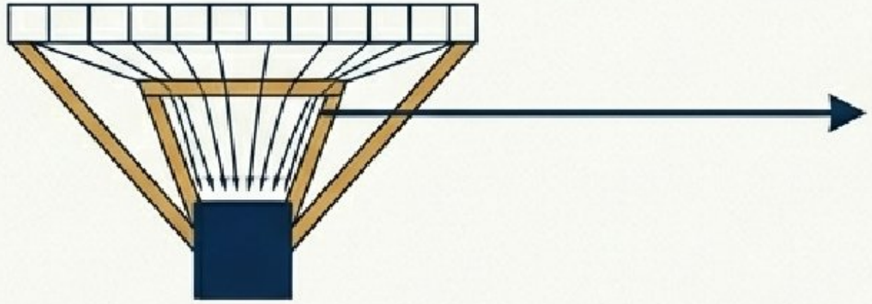
Mechanism 1: Conv2D



Filters for edges, corners, and foundational shapes.

Mechanism 2: MaxPooling2D





Compresses data, retaining only the strongest mathematical signals to save processing power.

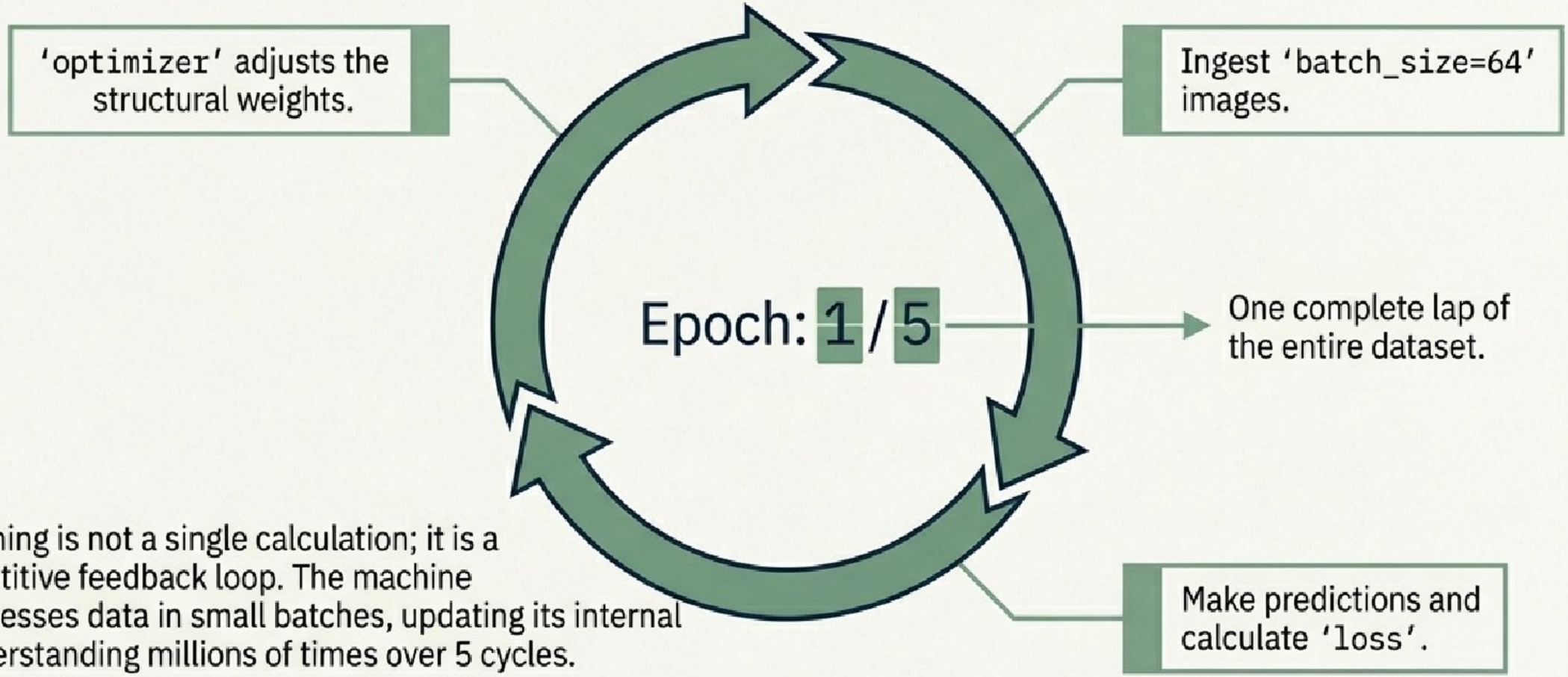
Defining the Rules: Model Compilation

```
model.compile(optimizer='adam', loss='sparse_categorical_crossontropy', metrics=['accuracy'])
```

The Optimization Trio		
1	2	3
The Navigator `optimizer='adam'`	The Penalty `loss='...crossentropy'`	The Scorecard `metrics=['accuracy']`
Updates the network's internal weights based on past mistakes. Finds the fastest route to high accuracy.	Mathematically calculates exactly how far off the AI's prediction was from the true label.	Translates complex loss mathematics into a human-readable format (e.g., 95% correct).

The Iterative Loop: Training the Engine

```
model.fit(x_train, y_train, epochs=5, batch_size=64)
```

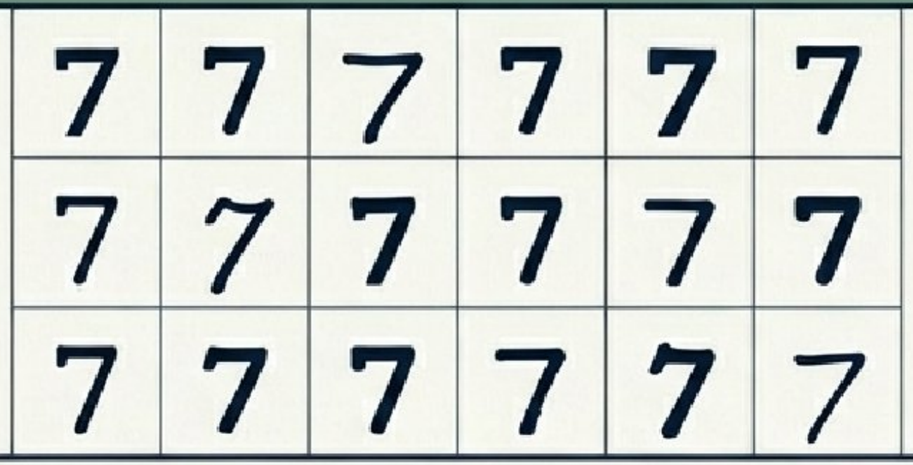


Training is not a single calculation; it is a repetitive feedback loop. The machine processes data in small batches, updating its internal understanding millions of times over 5 cycles.

Testing for Generalization

```
test_loss, test_acc = model.evaluate(x_test, y_test)
```

The Training Arena



The AI studying familiar, memorized examples.

The Real World

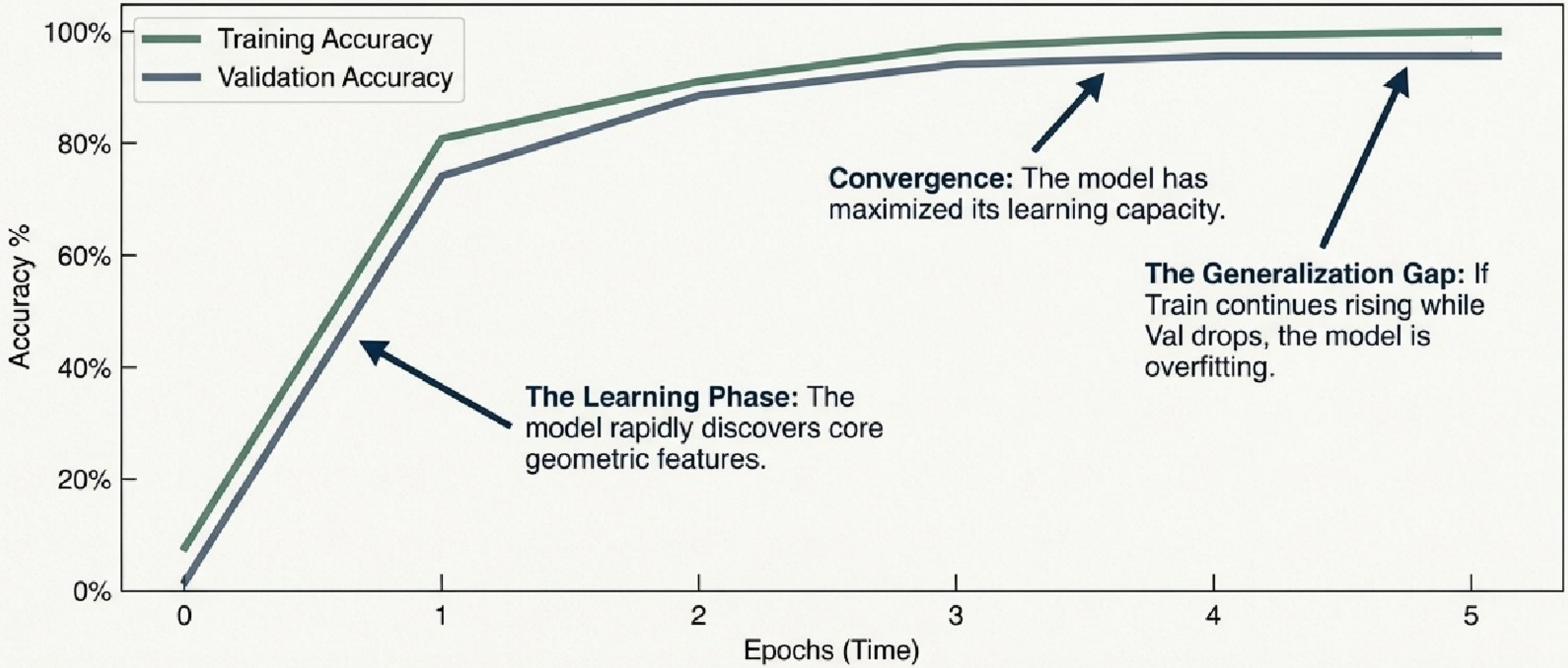
The AI encountering pristine, untouched data.

Final Test Accuracy: 98.2%

True intelligence is not memorization; it is generalization. ‘model.evaluate’ forces the network to prove its capabilities on unseen data to ensure it hasn’t just memorized the training answers.

Diagnostic Visualization: Reading the Learning Curve

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
```



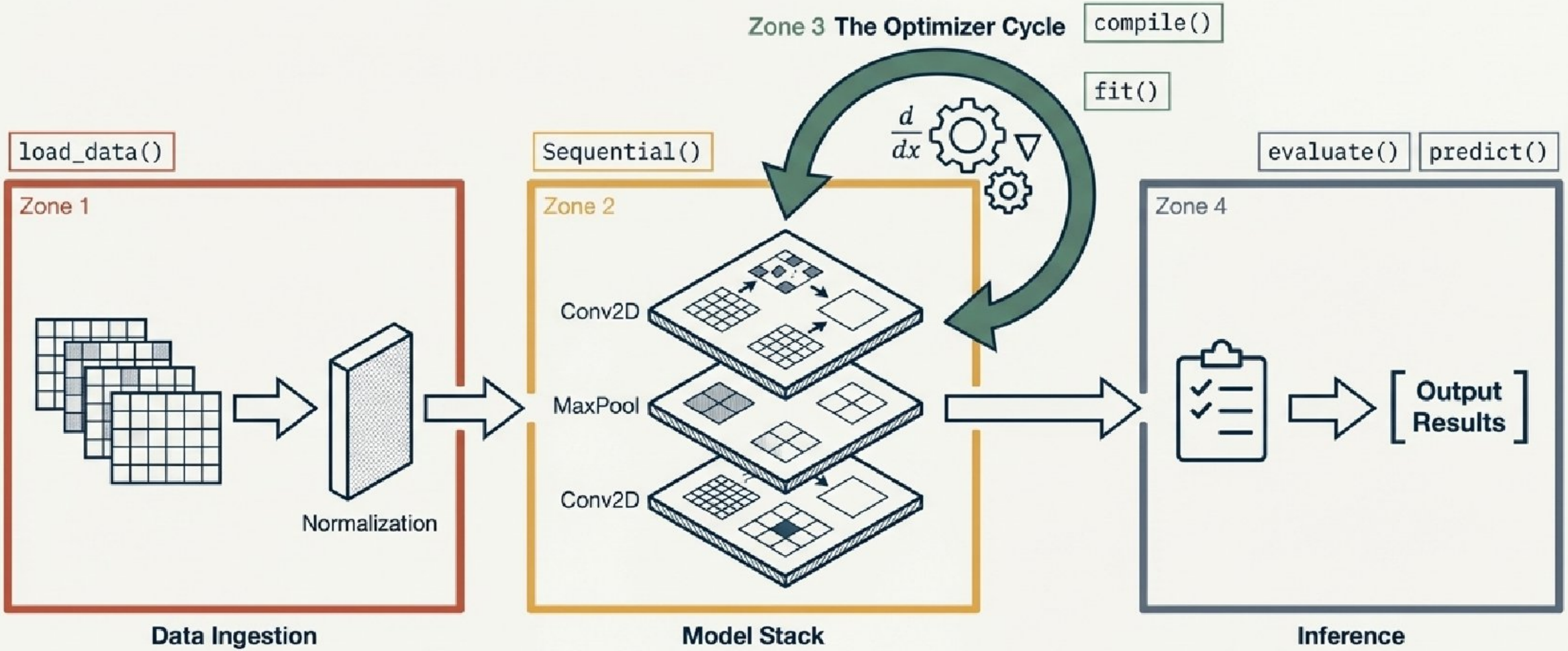
Deployment: Running Predictions

```
predict_multiple_random_images(5)
```

The pipeline is complete. The network requires no further weight adjustments or labels. It accepts a raw array of normalized pixels and instantaneously outputs a mathematically confident categorization.

The Complete Circuit

The Python syntax acts as the remote control for this physical engine.



Every function serves the greater architecture. Data ingestion feeds the structure; compilation defines the rules; training turns the gears; and evaluation proves the result.